Informatik I **Wintersemester 2025/26**
Dr. Henrik Brosenne Georg-August-Universität Göttingen
Paul Sarnighausen-Cahn Institut für Informatik

Übungsblatt 02

Allgemeine Information zum Bestehen der Prüfungsvorleistung.
Um die Prüfungsvorleistung zu bestehen, müssen Sie auf jedem Übungszettel mindestens 40% der Punkte in Theorie und Praxis erreichen. Es gibt jedoch zwei Ausnahmen: Sie dürfen auf zwei Zetteln bei den theoretischen Aufgaben und auf zwei Zetteln beim praktischen Teil unter die 40%-Grenze fallen, ohne dass dies zum Nichtbestehen führt.
Das bedeutet konkret: Wenn Sie beispielsweise auf einem Übungsblatt weniger als 40 % in der Theorie erreichen, können Sie den praktischen Teil trotzdem bestehen – und umgekehrt.

Theoretische Übungen

Abgabe bis Di., 18.11., 14 Uhr

Allgemein Die Lösung dieser Aufgaben wird über VIPS abgegeben. Bitte geben Sie für den Theorieteil eine PDF-Datei mit Ihren Lösungen ab.
Formatieren Sie Ihre Abgabe so, dass die Aufgaben klar durch einzelne Überschriften voneinander getrennt sind.

Aufgabe 1 – 15 Punkte

Algorithmus (i1-u-sm0)

Gesucht ist ein Algorithmus zum Quadrieren positiver ganzer Zahlen.

Problemspezifikation

- Gegeben: Eine positive ganze Zahl `x`
- Gesucht: Das Quadrat $\texttt{x}^2$
- Erlaubte Operationen: Großes Eins-plus-Eins (nur Addition $\texttt{a} + \texttt{b}$ für `int`-Werte $\texttt{a}, \texttt{b}$ und $\texttt{a}, \texttt{b} \geq 0$).

Idee

Das Quadrat einer positiven ganzen Zahl `x` ist die Summe der ersten `x` ungeraden positiven ganzen Zahlen beginnend bei Eins.

Aufgabe

Entwickeln und formulieren Sie einen einfachen Algorithmus, präzisiert als Python Code. (15 Punkte)

Aufgabe 2 – 15 Punkte

Binomialkoeffizient (i1-u-gr0)

Das unten beschriebene Programm in Pseudocode wird mit den Werten $n = 10$ und $k = 7$ gestartet.

Erstellen Sie ein Ablaufprotokoll (siehe Skript) des Programms indem Sie nach jedem Durchlauf der Schleife die aktuellen Belegungen der Variablen notieren.

```
procedure binomialkoeffizientEffizient
Input: Dezimalzahlen n, k
ergebnis := n - k + 1
i := 2
while i <= k do
    ergebnis := ergebnis * (n - k + i)
    ergebnis := ergebnis / i
    i := i + 1
return ergebnis
```

(15 Punkte)

Aufgabe 3 – 20 Punkte

Schleifeninvarianten (i1-u-si0)

In dieser Aufgabe sollen Sie (informell) Schleifeninvarianten bestimmen.

```
i = 0
summe = 0
while i < 5:
    summe = summe + i
    i = i + 1
```

Finden Sie eine Aussage über *summe* und i, die in jeder Iteration wahr ist.
Überprüfe diese Vermutung mit einer Trace-Tabelle. (10 Punkte)

```
x = 10
count = 0
while x > 1:
    x = x // 2
    count = count + 1
```

Beschreibe eine Invariante, die die Beziehung zwischen x und *count* erklärt.
Was tut das Programm insgesamt?
(10 Punkte)

Praktische Übung

Abgabe der Prüfsumme bis Di., 18.11., 14 Uhr
Testat ab Di., 18.11., 18 Uhr

Hilfe zum Bearbeiten der praktischen Übungen können Sie grundsätzlich jeden Tag in den Rechnerübungen bekommen.

- Bearbeiten Sie die praktischen Übungen in dem bereitgestellten Jupyter Notebook.
- Um die praktische Übung testieren zu lassen, **müssen** Sie einen Termin in der zugehörigen Rechnerübung reservieren. Ein Testat ohne Termin ist nicht möglich.
- Übermitteln Sie die Prüfsumme über Vips
- Öffnen Sie die Terminvergabe für diese praktische Übung und lassen Sie den von Ihnen reservierten Termin anzeigen.
- Zeigen Sie ihr Notebook und führen Sie alle zellen aus, damit am Ende die Prüfsumme angezeigt wird.
- Der CIP-Pool befindet sich hier: `https://lageplan.uni-goettingen.de/?ident=2412_1_1.UG_-1.110`

Aufgabe 1 – 15 Punkte

Markdown und AsciiMath (i1-u-ha0)

Einfacher Text (*plain text*) besteht nur aus Zeichen eines feststehenden Zeichensatzes (z.B. ASCII, ISO-Latin-15, UTF-8). Die Gliederung eines einfachen Textes ist nur durch Steuerzeichen möglich (z.B. Tabulator, Zeilenumbruch, Seitenwechsel), die im Zeichensatz enthalten sind. Schriftzeichen verschiedener Sprachen oder Symbole (z.B. Klammern, Operatoren aus Arithmetik oder Mengenlehre) stehen nur zur Verfügung, wenn der Zeichensatz diese enthält. Formatierung (z.B. Fett, Kursiv, Hochstellen, Tiefstellen) ist nicht möglich.

Eine Auszeichnungssprache (*markup language*) kann für die Gliederung und Formatierung von einfachem Text verwendet werden. Weiterhin ist es möglich, nicht im Zeichensatz enthaltene Schriftzeichen und Symbole zu kodieren. Dazu werden ausschließlich Zeichenfolgen des Zeichensatzes verwendet.

Für die Gliederung und Formatierung von einfachen Texten kann in Jupyter Notebooks eine Markdown Zelle erstellt werden, in der man mathematische Symbole, Coe Einbettungen und überschriften schön formattieren kann. Für eine Einleitung in Jupyter Markdown, siehe: `https://jupyter-notebook.readthedocs.io/en/stable/examples/Notebook/Working%20With%20Markdown%20Cells.html#Markdown-Cells` oder auch: `https://daringfireball.net/projects/markdown/`

Beispiel

```
# Informatik I

Henrik Brosenne, Wintersemester 2025/26

## 1.2 Problem, Algorithmus, Programm

### 1.2-2 Gute und schlechte Spezifikationen

**Erster Schritt zur Lösung eines Problems ist eine gute Spezifikation:**

- unzweideutig,
- vollständig und
- (idealerweise) einfach und kurz

*Beispiel für unzureichende Spezifikation*

**gesucht** Lösungen der Gleichung $a_2 * x^2 + a_1 * x + a_0 = 0$

Welche Größen sind gegegeben? Was sind die Wertebereiche?
Wie ist mit Sonderfällen (z.B. keine reelle Lösung) zu verfahren?

*Besser:*

**gegeben** drei Koeffizienten $a_0, a_1, a_2 \in \mathbb{R}$

**gesucht** alle reellen oder komplexen Werte $x$
mit $a_2 * x^2 + a_1 * x + a_0 = 0$
```

Bemerkungen

- Ein Absatz besteht aus einer oder aus mehreren Textzeilen, Markdown ignoriert Zeilenumbrüche in Absätzen, d.h. damit Gliederungen, z.B. Listen, erkannt werden können, müssen diese mit einer Leerzeile eingeleitet werden.
- Um mathematische Symbole einzubetten, werden $ Zeichen verwendet. Um eine Vorstellung zu bekommen was alles möglich ist, können Sie diesem link folgen: `https://jupyter-notebook.readthedocs.io/en/stable/examples/Notebook/Typesetting%20Equations.html`

Aufgabe

Schreiben Sie eine Markdown Zelle in dem bereitgestellten Juypter Notebook die mit folgendem Text übereinstimmt.

Informatik I, Übung 02, Aufgabe 1

Markdown und LaTeX

Größter gemeinsamer Teiler (ggT)

Das ggT-Problem. **Gegeben:** Zwei positive ganze Zahlen $a, b \in \mathbb{Z}$ mit $a, b > 0$.

Gesucht: Eine positive ganze Zahl $d \in \mathbb{Z}$ mit $d > 0$ und folgenden Eigenschaften:

- d teilt a und b ohne Rest,
- es gibt **keine** ganze Zahl $d' \in \mathbb{Z}$ mit $d' > d$, die a und b ohne Rest teilt.

Der euklidische Algorithmus (klassisch)

Eingabe: Zwei ganze Zahlen a, b mit $a > b \geq 0$.

Initialisierung:

$$x \leftarrow a, \quad y \leftarrow b$$

Iteration: Wiederhole Folgendes, solange $y \neq 0$:

- Falls $x > y$, dann $x \leftarrow x - y$,
- sonst $y \leftarrow y - x$.

Abschluss:

$$d \leftarrow x$$

Rückgabe: d

Hinweis: $A \leftarrow B$ bedeutet: speichere den aktuellen Wert von B in A.

Bemerkung.
Die Formatierung muss nicht exakt übereinstimmen. Es reicht den Inhalt korrekt wiederzugeben.
Den Pfeil nach links wird durch den Befehl

`\leftarrow`

erzeugt.
(15 Punkte)

Aufgabe 2 – 20 Punkte

Automatische Trace-Tabelle (i1-p-pa0)

Laden Sie das in der Stud.IP-Veranstaltung *Informatik I* unter *Dateien→Übung→uebung02* hinterlegten Jupyter Notebook auf Ihren Rechner. Die Datei enthält Algorithmen, die das ggT-Problem mithilfe der Euklidischen Algorithmen aus der Vorlesung lösen.

`EuklidKlassisch`

```
# ggT(x,y)                                          // K1
while  y != 0:                                        // K2
    if  x > y:                                        // K3
                x = x - y                                     // K4
    else:
                y = y - x                                  // K5

print(x)                                 // K6
```

`EuklidModern`

```
# ggT(x,y)                                         // M1
while  y != 0:                                       // M2
    r = x % y                                        // M3
    x = y                                            // M4
    y = r                                            // M5

print(x)                                 // M6
```

Versehen Sie die Quelltexte `Euklid Klassisch` und `Euklid Modern` mit `print`-Anweisungen, sodass eine einfache Trace-Tabelle (Ablauf-Protokoll), entsprechend der nachfolgenden Beispiele, *von selbst* entsteht. Berücksichtigt werden müssen nur die mit `//K` und `//M` gekennzeichneten Zeilen.
(20 Punkte)

Beispiel

```
> Euklid Klassisch 15 35
| line  | x     | y     | comment
| K1    | 15    | 35    | ggT(15, 35)
| K2    | 15    | 35    |   while(35 != 0)
| K3    | 15    | 35    |     if(15 > 35)
| K5    | 15    | 20    |       y = 35 - 15
| K2    | 15    | 20    |   while(20 != 0)
| K3    | 15    | 20    |     if(15 > 20)
| K5    | 15    | 5     |       y = 20 - 15
| K2    | 15    | 5     |   while(5 != 0)
| K3    | 15    | 5     |     if(15 > 5)
| K4    | 10    | 5     |       x = 15 - 5
| K2    | 10    | 5     |   while(5 != 0)
| K3    | 10    | 5     |     if(10 > 5)
| K4    | 5     | 5     |       x = 10 - 5
| K2    | 5     | 5     |   while(5 != 0)
| K3    | 5     | 5     |     if(5 > 5)
| K5    | 5     | 0     |       y = 5 - 5
| K2    | 5     | 0     |   while(0 != 0)
| K6    | 5     | 0     | = 5
```

Beispiel

```
> Euklid Modern 15 35
| line  | x     | y     | r     | comment
| M1    | 15    | 35    | -     | ggT(15, 35)
| M2    | 15    | 35    | -     |   while(35 != 0)
| M3    | 15    | 35    | 15    |     r = 15 % 35
| M4    | 35    | 35    | 15    |     x = 35
| M5    | 35    | 15    | 15    |     y = 15
| M2    | 35    | 15    | 15    |   while(15 != 0)
| M3    | 35    | 15    | 5     |     r = 35 % 15
| M4    | 15    | 15    | 5     |     x = 15
| M5    | 15    | 5     | 5     |     y = 5
| M2    | 15    | 5     | 5     |   while(5 != 0)
| M3    | 15    | 5     | 0     |     r = 15 % 5
| M4    | 5     | 5     | 0     |     x = 5
| M5    | 5     | 0     | 0     |     y = 0
| M2    | 5     | 0     | 0     |   while(0 != 0)
| M6    | 5     | 0     | -     | = 5
```

Hinweis

Zu Formatierung der Aufgabe können Sie das Tabulatorzeichen `\t` verwenden.

Aufgabe 3 – 15 Punkte

Einfache und iterierte Quersumme (i1-p-lo0)

Die Quersumme einer Zahl ist die Summe ihrer Ziffern.
Beispiel. Die Quersumme von 99 ist 18 ($9 + 9 = 18$).

Die iterierte Quersumme erhält man, indem man so lange wiederholt die Quersumme bildet, bis das Ergebnis einstellig ist.
Beispiel. Die iterierte Quersumme von 99 ist 9 ($99 \to 9 + 9 = 18 \to 1 + 8 = 9$).

1. Schreiben Sie eine Applikation, die von einer natürlichen Zahl die Quersumme berechnet und ausgibt.

   Lassen Sie die Quersumme der Zahlen 42, 3678 und 6947859 berechnen.
   (5 Punkte)

2. Schreiben Sie auf Grundlage der Applikation aus Aufgabenteil 1 eine Applikation, die für eine natürliche Zahl die iterierte Quersumme einer Zahl berechnet und ausgibt.

   Testen Sie Ihr Programm mit den Zahlen 24, 8763 und 9587496.
   (10 Punkte)

Hinweise

- Diese Aufgabe ist Grundlage einer Aufgabe (gewichtete, alternierende Quersumme) der nächsten praktischen Übung.